

Chap.5 R-CNN (Regions with CNN)

방 수 식 교수
(bang@tukorea.ac.kr)

한국공학대학교 전자공학부

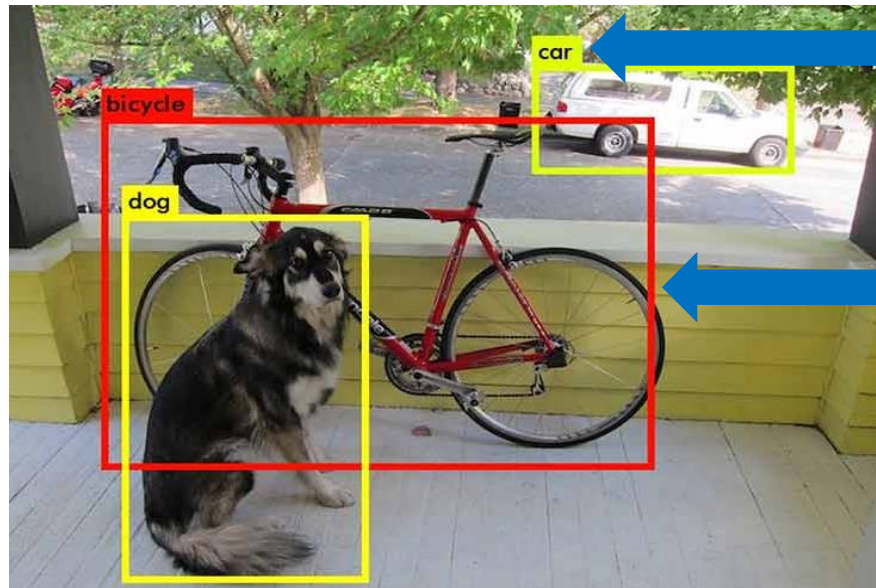
2024년도 2학기
고급머신러닝실습 & 인공지능설계실습2

Object Detection



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

- **Classification(분류)이란?**
 - 어떤 물체가 있을 때, 이 물체가 어떤 **Class**에 해당하는지를 표현하는 것
- **Localization(지역화)이란?**
 - 해당 물체가 어디에 있는지 나타내는 것



Label
(Classification)

Bounding Box
(Localization)

- **Objection Detection의 목적**
 - Classification + Localization

Object Detection



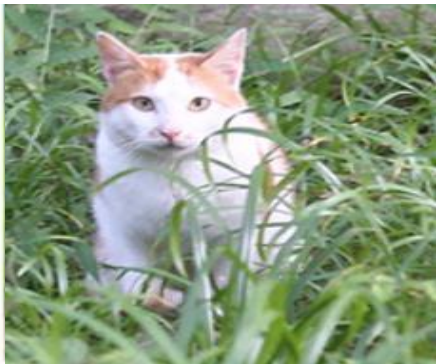
■ Object Detection이란?

- **Classification**과 **Localization** 둘 다 수행
- 물체의 위치와 라벨, 그리고 한 물체 뿐만 아니라 여러 물체가 있을 때 Multi object detection을 수행

■ Instance Segmentation이란?

- 물체들이 어떤 픽셀에 존재하는지 segmentation 하는 알고리즘

Classification



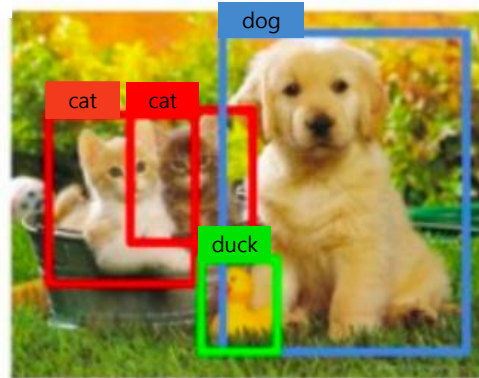
Cat

Classification
+Localization



Cat

Object
Detection



Cat, Duck, Dog

Instance
Segmentation

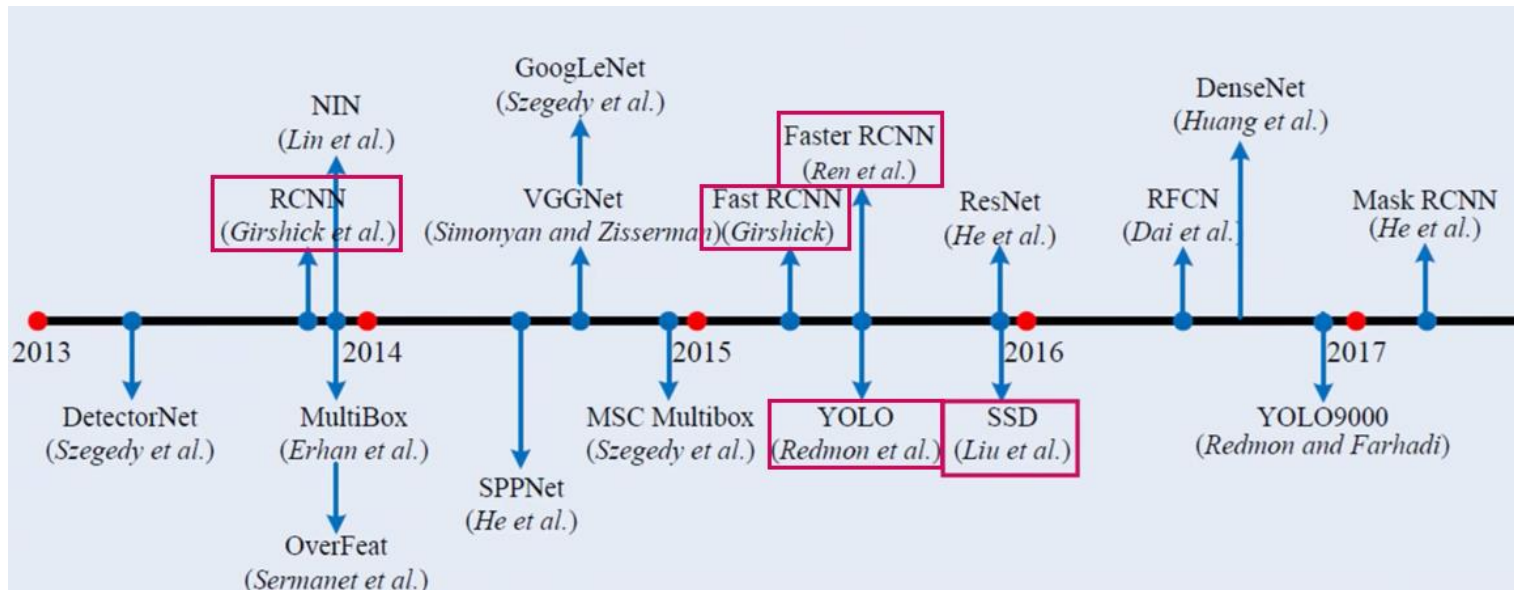


Cat, Duck, Dog

Object Detection (객체 검출)



- CNN 알고리즘을 활용하여 다양한 분야에서 활용됨
- 2-stage Detector와 1-stage Detector로 분류
 - 2-stage Detector는 전체 task를 두 단계로 나누어 순차적 실행 (RCNN 계열)
 - 1-stage Detector는 두 단계를 동시에 실행 (YOLO계열)



2-stage Detector 처리 속도 ↓, 정확도 ↑
1-stage Detector 처리 속도 ↑, 정확도 ↓

Object Detection (객체 검출)



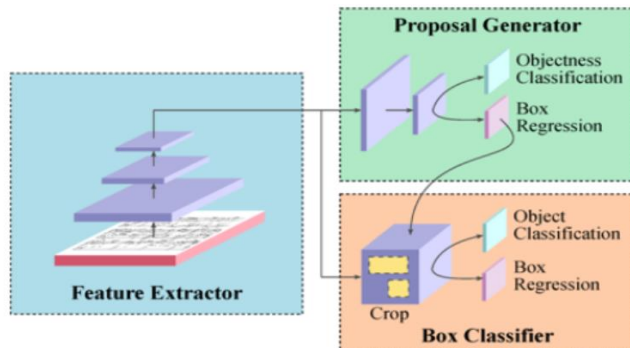
■ 2-stage Detector

- 물체의 위치를 찾는 Region Proposal과 물체를 분류하는 Region Classification 두 단계를 **순차적으로 진행**
- 물체가 있을 것 같은 영역을 Bounding Box로 찾은 후, Classification을 진행
- 두 단계를 순차적으로 진행하기 때문에 **처리 속도가 느림**

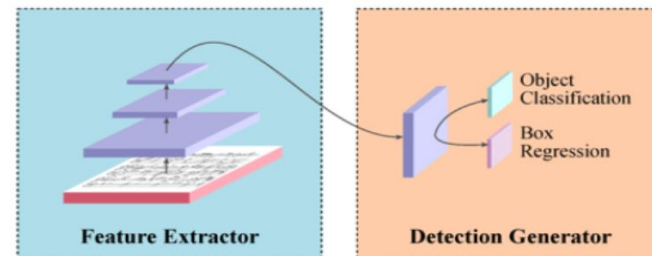
■ 1-stage Detector

- 2-stage Detector의 단점을 해결하기 위해 개발됨
- Region Proposal과 Region Classification을 **동시에 진행**

2-stage Detector



1-stage Detector



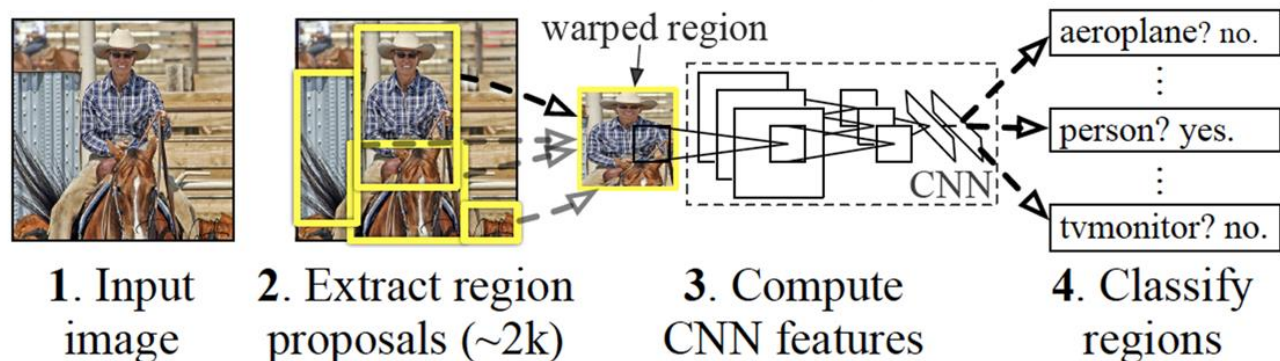
R-CNN(Regions with CNN)



▪ R-CNN(Regions with CNN)

- CNN을 Object Detection 분야에 최초로 적용시킨 모델
- 대표적인 2-stage Detector의 모델 중 하나
- 물체의 위치를 찾는 Region Proposal과
물체를 분류하는 Region Classification 두 단계를 순차적으로 진행

R-CNN: *Regions with CNN features*



Task: ➡

1. Region Proposal

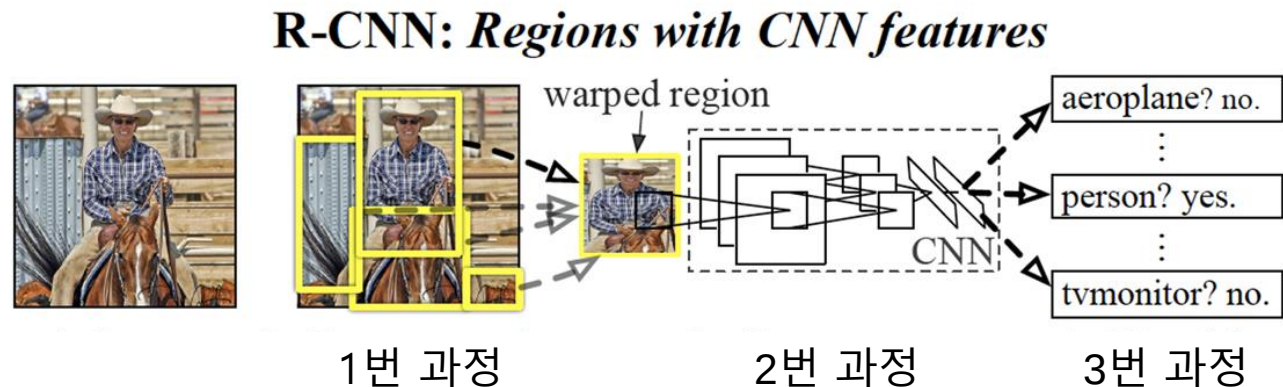
2. Region Classification

R-CNN(Regions with CNN)



■ R-CNN 수행과정

1. Region Proposal 하는 방법 중 **Selective Search**를 통해 **Object가 있을 것 같은 다수의 영역을 Bounding Box**로 만듦
2. 해당 영역들의 사이즈가 달라 CNN의 Input 사이즈를 맞추기 위해 **Warp** 진행
3. Warp된 이미지를 각각 **CNN**을 통해 분류



Task: →

1. Region Proposal

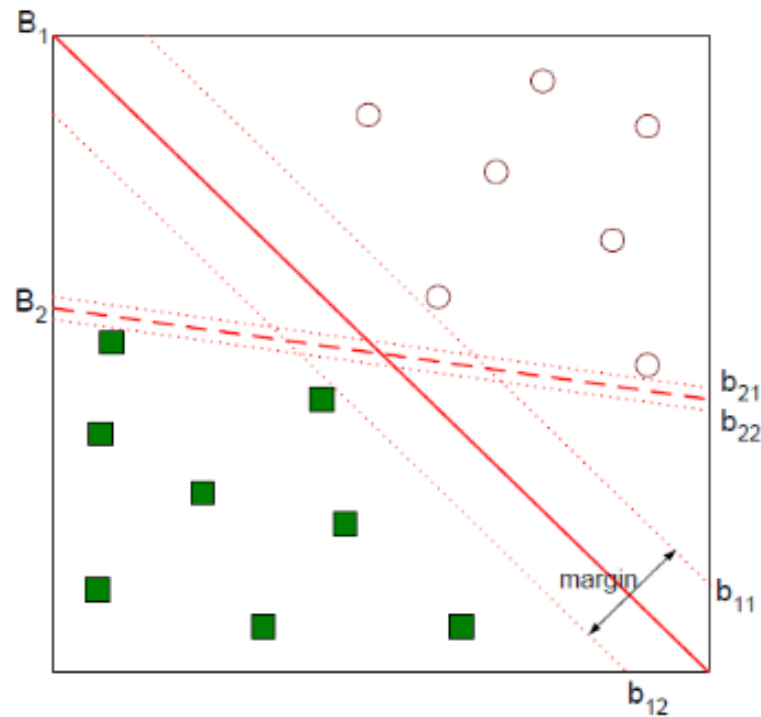
2. Region Classification

최초 논문에서는 분류 모델을 SVM로 제시

[참고] SVM (Support Vector Machine)



- 결정 경계를 제대로 만들자



■ Exhaustive Search

- 가능한 경우의 수를 일일이 나열하면서 답을 찾는 방법
- 효율성은 낮지만 알고리즘이 간단함
- Ex) 4자리 비밀번호 0000 ~ 9999 수를 순차적으로 맞을 때까지 입력

■ Segmentation

- 이미지의 유사한 부분을 픽셀별로 레이블화 시키는 것



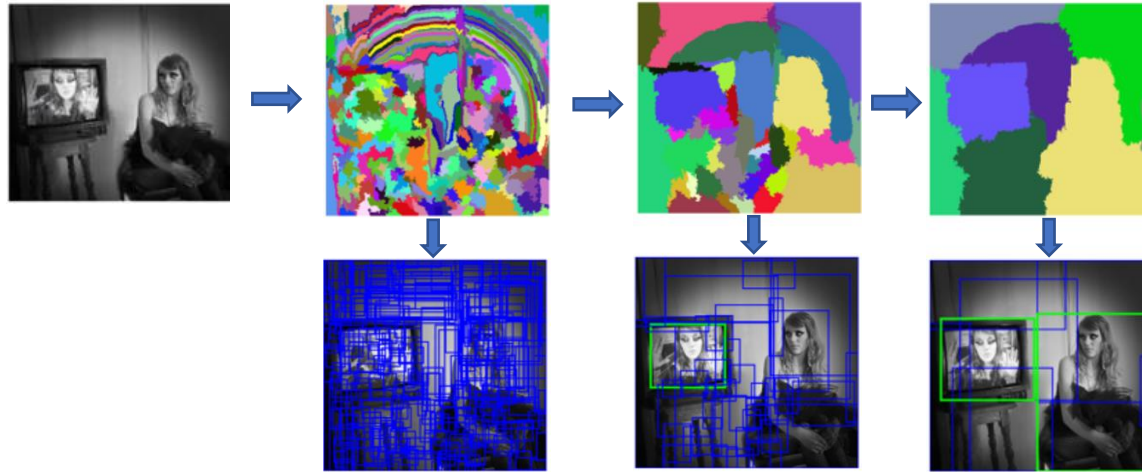
segmented →

- 1. Person
- 2. Purse
- 3. Plants
- 4. Sidewalk
- 5. Building

3	3	3	3	3	3	3	3	3	3	3	3	3	5	5	5	5	5	5
3	3	3	3	3	3	3	3	3	3	3	3	3	5	5	5	5	5	5
3	3	3	3	3	3	1	1	3	3	3	3	5	5	5	5	5	5	5
3	3	3	3	3	1	1	1	1	3	3	3	5	5	5	5	5	5	5
3	3	3	3	3	1	1	3	3	3	5	5	5	5	5	5	5	5	5
5	5	3	3	3	3	1	1	3	3	5	5	5	5	5	5	5	5	5
4	4	3	4	1	1	1	1	1	1	4	4	4	5	5	5	5	5	5
4	4	3	4	1	1	1	1	1	1	4	4	4	4	5	5	5	5	5
4	4	4	1	1	1	1	1	1	1	1	4	4	4	4	4	4	4	4
3	3	3	1	1	1	1	1	1	1	1	4	4	4	4	4	4	4	4
3	3	3	1	2	2	1	1	1	1	1	4	4	4	4	4	4	4	4
3	3	3	1	2	2	1	1	1	1	1	4	4	4	4	4	4	4	4

■ Selective Search이란?

- Segmentation방식 + Exhaustive Search방식을 결합
- 이미지에 물체가 있을 것 같은 영역을 모두 조사해 segmentation함.



■ Selective Search 순서

1. [초기화] ①Efficient graph-based image segmentation으로 초기 영역 결정
2. ②Greedy알고리즘을 통해 유사한 영역 병합
3. 통합된 영역들을 바탕으로 후보 영역 추출

Efficient graph-based image segmentation



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

■ Efficient graph-based image segmentation

1. Graph 형성

- V (vertex): 이미지의 각 픽셀
- E (Edge): 픽셀 간의 유사성을 의미하는 **weight(가중치)**

$$w(v_i, v_j) = \sqrt{(p_i - p_j)^2}$$

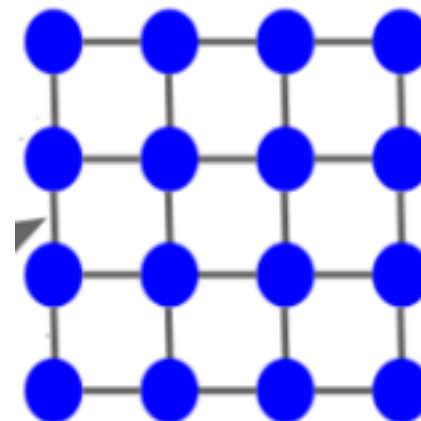
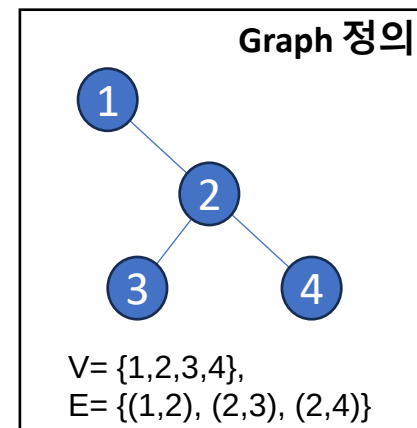
p: 픽셀 벡터 $p = [x, y, r, g, b]$
서로 유사하면 가중치 값은?

2. Edge 정렬

- Ascending order(오름차순)으로 정렬

3. 초기 Cluster 지정

- 각 Vertex를 모두 각자의 Cluster로 지정



Efficient graph-based image segmentation

4. Cluster 분리 및 통합

- 그룹 간 Edge 최소값 vs 그룹 내 Edge 최소값

True => 분리? 병합?

$$D(C_1, C_2) = \begin{cases} \text{true} & \text{if } Dif(C_1, C_2) > MInt(C_1, C_2) \\ \text{false} & \text{otherwise} \end{cases}$$



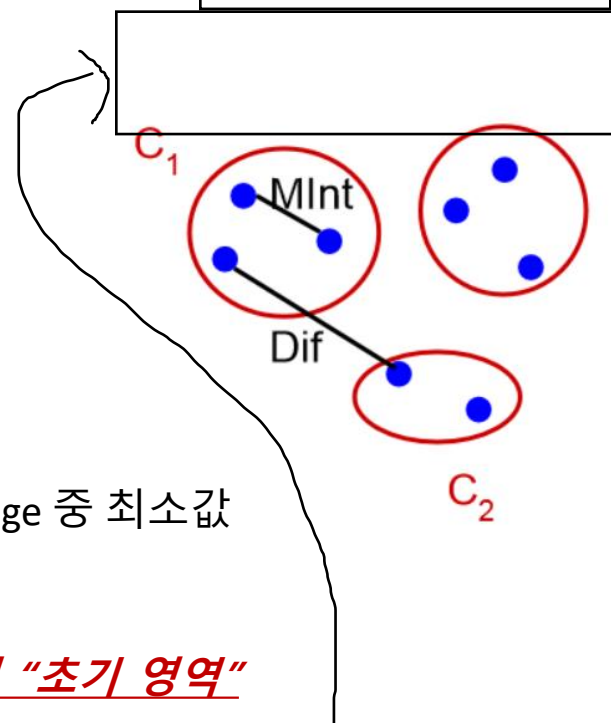
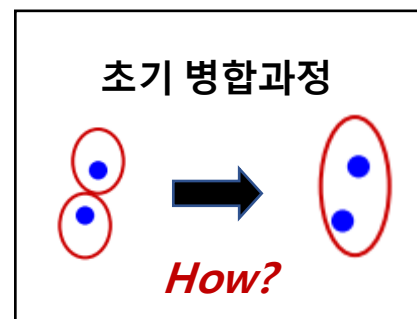
❖ $Dif(C_1, C_2)$: 두 클러스터를 서로 연결하는 Edge 중 최소값

- 두 클러스터 간의 차이

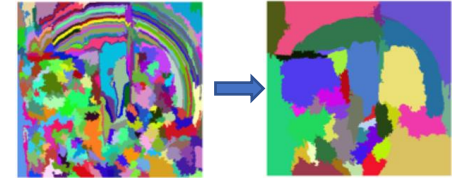
❖ $MInt(C_1, C_2)$: 각각의 클러스터 내에서 Vertex를 연결하는 Edge 중 최소값

- 클러스터 내부 차이

5. 4. 반복 수행 => **반복 수행한 결과 : Selective Search의 "초기 영역"**



GREEDY 알고리즘



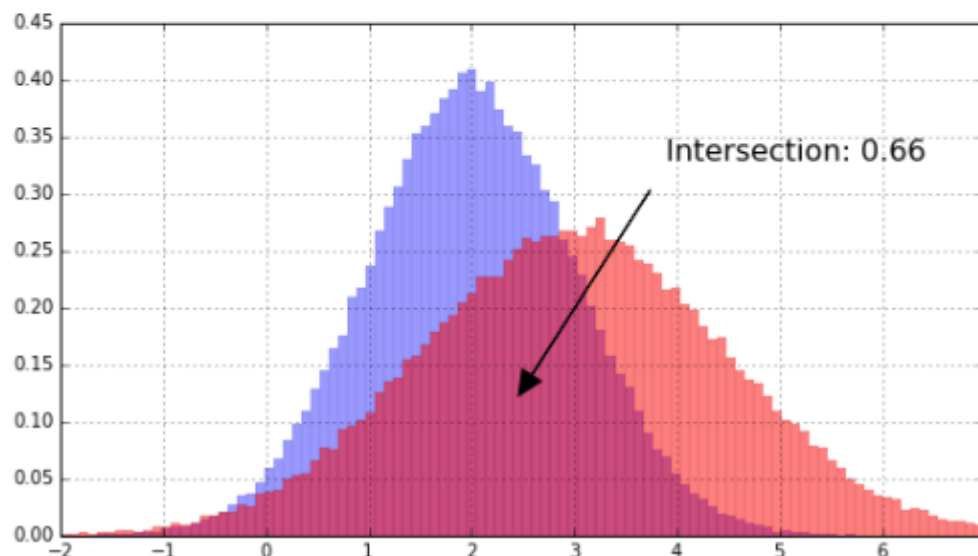
1. 이미지의 영역 초기화하고 영역 List 생성
2. 영역 List 내 각 영역 간의 유사도 계산
 - 유사도 계산 시 두 영역 간의 Color, Size, Texture, fill값이 고려됨.
3. 가장 유사한 영역 2개를 1개로 병합
4. 병합되기 전 영역 2개를 List에서 삭제하고 병합된 영역을 List에 추가
5. 반복 및 종료
 - 단계 2~4을 반복함
 - 모든 초기 영역이 하나의 덩어리로 결합된 후까지 과정을 반복하거나, 사전에 정의된 종료 조건(예: 생성된 영역 개수, threshold 등)을 충족할 때까지 진행

[참고] 유사도 계산에서 이용된 값



■ Color (컬러 분포)

- 각 컬러 채널을 25개 bin으로 설정
- 각 region 마다 컬러 히스토그램 생성 $C_i = c_i^1, \dots, c_i^n$
- 차원 수 $n = 75$ (RGB 3채널 $\times$ 25개의 Bin)
- L_1 norm 정규화 $[0, 1]$
- 인접한 regions의 교집합을 유사도로 측정
- $s_{colour}(r_i, r_j) = \sum_{k=1}^n \min(c_i^k, c_j^k)$
- 유사성 척도의 min function은 두 히스토그램의 교집합을 나타냄



[참고] 유사도 계산에서 이용된 값



■ Texture (주변 픽셀값에 대한 변화량)

- $\sigma = 1$ 인 8방향의 가우시안 미분을 적용
- 10개의 Bin으로 히스토그램 도출 $T_i = t_i^1, \dots, t_i^n$
- L_1 norm 정규화 $[0, 1]$
- 80차원(8방향 $\times$ 10차원의 Bin)의 벡터로 인접한 region들의 유사성을 평가
- RGB(3차원)의 경우: 8방향 $\times$ Bin의 수(10차원) $\times$ RGB(3차원) = 240차원
- $s_{texture}(r_i, r_j) = \sum_{k=1}^n \min(t_i^k, t_j^k)$

■ Size (영역의 사이즈)

- 사이즈가 작을 수록 유사도가 높음
- $s_{size}(r_i, r_j) = 1 - \frac{size(r_i) + size(r_j)}{size(im)}$ **영역의 Size는 어떻게 구할까?**
- im 은 원 이미지를 나타냄

■ Fill: 후보 Box와의 크기 차이

- candidate Bounding Box와 Region들의 사이즈의 차이가 적을수록 유사도가 높음
- $s_{fill}(r_i, r_j) = 1 - \frac{size(BB_{ij}) - size(r_i) - size(r_j)}{size(im)}$
- im 은 원 이미지를 나타냄



■ Warp

- 이미지의 특정 영역을 변형시키는 과정
- Selective Search를 통해 나온 다양한 사이즈, 비율의 Bounding Box를 특정 크기로 통일시키는 과정

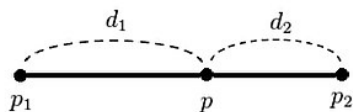


■ Interpolation(보간)

- 알려진 값 사이(중간)에 위치한 값을 추정하는 것

■ 1D linear interpolation(선형 보간법)

- 두 지점 사이의 값을 추정할 때, 두 지점과의 직선 거리를 선형적으로 결정



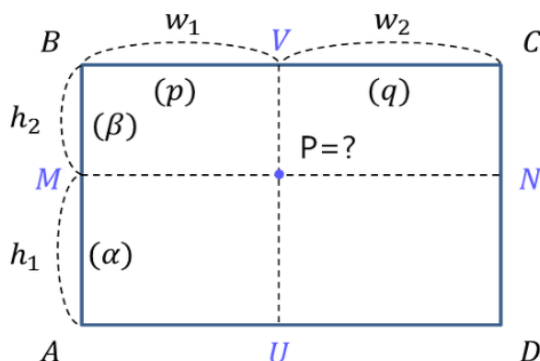
p_1, p_2 의 데이터값이 $f(p_1), f(p_2)$ 일때, $f(p) = \frac{d_2 p_1 + d_1 p_2}{d_1 + d_2}$

[참고] 2D Bilinear interpolation(이중선형 보간법)



■ 2D Bilinear interpolation(이중선형 보간법)

- 1차원에서 2차원으로 확장한 것 이며, 선형 보간을 바탕으로 수행.
- Nearest neighbor interpolation에 비하여 더 Smooth함.
- 픽셀당 linear interpolation 을 여러 번 수행하여, 많은 계산 량을 가짐.

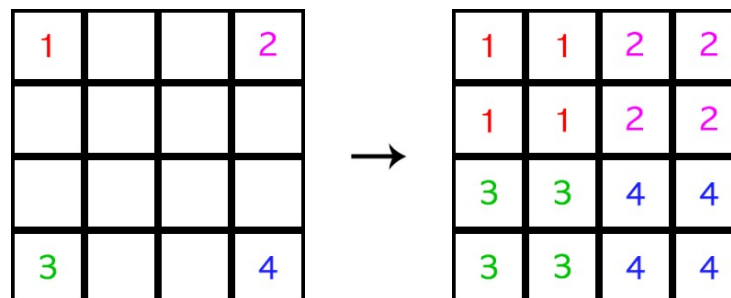


$$P = q(\beta a + \alpha B) + p(\beta d + \alpha C)$$

$$\alpha = \frac{h_1}{h_1 + h_2}, \beta = \frac{h_2}{h_1 + h_2}, p = \frac{w_1}{w_1 + w_2}, q = \frac{w_2}{w_1 + w_2}$$

■ Nearest neighbor interpolation

- 가장 가까운 픽셀값을 사용해 보간.



■ CNN 모델 저장 및 불러오기

- Chap.4에서 학습시킨 CNN 모델을 저장 및 불러오기

```
#자신만의 CNN 모델
model = keras.models.Sequential([
    '''CNN 모델'''
])

#CNN 모델 학습
history = model.fit('''CNN 학습''')

#학습된 모델 저장
model.save('mnist_cnn.h5')
```

Chap.4에서 진행한
CNN 학습



 mnist_cnn.h5
모델 파일이 저장됨

학습된 모델 저장

다른 python 파일에서 불러올 수 있음

```
1 from tensorflow import keras
2
3 mnist = keras.datasets.mnist
4 (X_train_full, Y_train_full), (X_test, Y_test) = mnist.load_data()
5
6 new_model = keras.models.load_model('mnist_cnn.h5')
7 new_model.evaluate(X_test, Y_test)
```

```
313/313 [=====] - 1s 2ms/step - loss: 0.0390 - accuracy: 0.9890
```

■ R-CNN 실습

- 필요 라이브러리 설치 및 데이터 불러오기



가상환경 진입



```
pip install opencv-python
```

```
pip install selectivesearch
```

자신의 가상환경에 설치!!

```
import numpy as np
import matplotlib.pyplot as plt
import selectivesearch
import cv2
import os

img_dir = 'MNIST\\data\\images\\'
img_list = os.listdir(img_dir) #해당 폴더에 있는 파일 리스트

i = 0 #첫번째 데이터에 대해
img = cv2.imread(img_dir + img_list[i]) #이미지
temp_img = img.copy()
```

반복문을 활용하여
데이터 전체를 불러올 수 있음

■ R-CNN 실습

- Selective Search 사용

```
#selective search 실행
```

```
_, regions = selectivesearch.selective_search(img, scale=800,  
                                              min_size=1200)
```

```
#regions 값 중 rect 값만 추출
```

```
region_rects = [cand['rect'] for cand in regions]  
region_rects = np.array(region_rects)
```

scale : object의 대략적인 크기
min_size : 최소 BB 크기

```
bbox = []
```

```
bboxes = []
```

```
for rect in region_rects:
```

```
    xmin = rect[0]
```

```
    ymin = rect[1]
```

```
    xmax = xmin + rect[2]
```

```
    ymax = ymin + rect[3]
```

```
    bbox = [(xmin, ymin, xmax, ymax)]
```

```
    bboxes += bbox
```

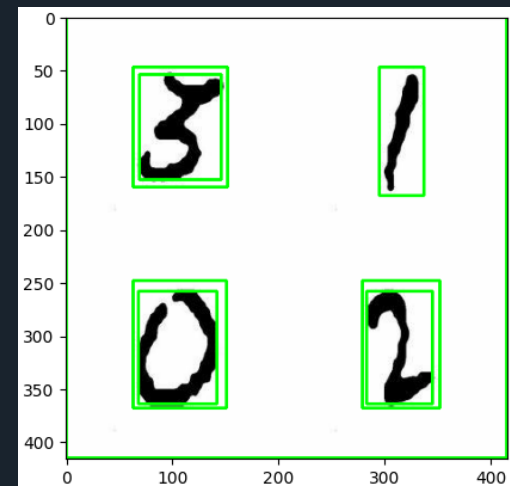
```
bboxes = np.array(bboxes)
```

```
for i, cands in enumerate(bboxes):
```

```
    bb = cands
```

```
    temp_img = cv2.rectangle(temp_img, (bb[0], bb[1]), (bb[2], bb[3]),  
                             color=(0, 255, 0), thickness=2)
```

```
plt.imshow(temp_img)
```



Selective Search 결과

■ R-CNN 실습

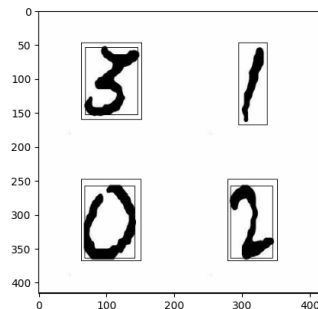
- Bounding Box를 따라 이미지 Warp

```
test_img = []
for i in range(len(bboxes)):
    #바운딩 박스 영역 잘라내기
    cropped_img = temp_img[bboxes[i][1]:bboxes[i][3], bboxes[i][0]:bboxes[i][2]]

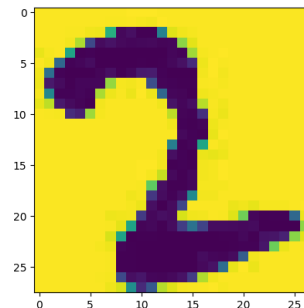
    #자른 이미지를 28by28로 warp / 선형보간법 사용
    warped_img = cv2.resize(cropped_img, (28,28), interpolation= cv2.INTER_LINEAR)

    #CNN 모델에 넣기 위해 흑백 형식으로 변환
    warped_img_gray = cv2.cvtColor(warped_img, cv2.COLOR_BGR2GRAY)

    #test_img에 추가
    warped_img_gray = np.expand_dims(warped_img_gray, axis = -1)
    test_img.append(warped_img_gray)
```



Selective Search 결과



warping된 image 중 1개

■ R-CNN 실습

- Warping된 이미지를 CNN 모델을 통해 분류

```
from tensorflow import keras

CNN_model = keras.models.load_model('mnist_cnn_1.h5')

test_img = np.array(test_img)

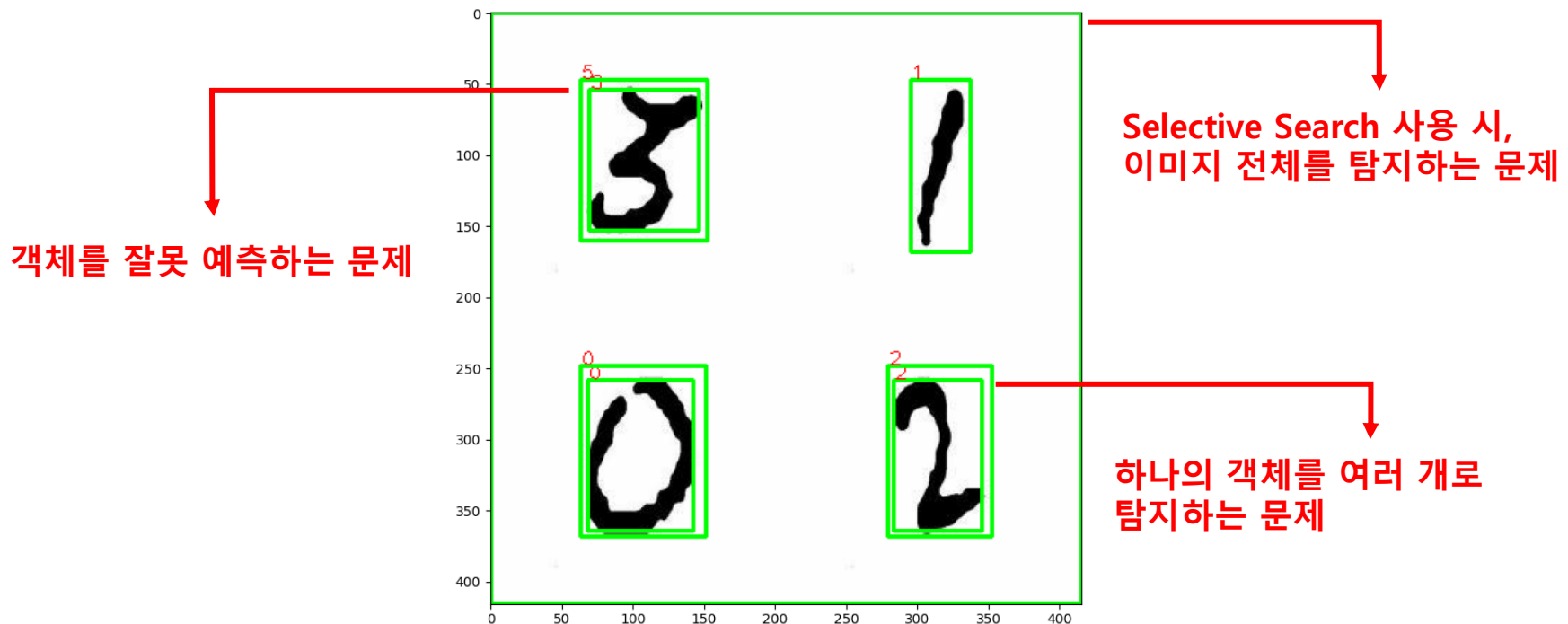
#모델 예측
y_pred_prob = CNN_model.predict(test_img)
y_pred = np.argmax(y_pred_prob, axis=1)

temp_img = img.copy()
for i,cands in enumerate(bboxes):
    bb = cands
    temp_img = cv2.rectangle(temp_img, (bb[0],bb[1]),(bb[2],bb[3]),color=(0,255,0), thickness=2)
    #예측값 함께 출력
    temp_img = cv2.putText(temp_img, str(y_pred[i]), (bb[0],bb[1]),1,1,color=(255,0,0))
plt.imshow(temp_img)
```

■ R-CNN 실습과정 정리

- Selective Search를 통해 찾은 Bounding Box를 warp
- 자신만의 MNIST 데이터를 분류하는 CNN모델을 설계 후 학습
- CNN모델을 이용해 Class 분류

■ R-CNN 실습결과



각 실습문제에 대해 code(+ 주석), 그래프, 분석 내용 등은 필수적으로 포함시킬 것

실습 #1) R-CNN 실습 과정을 따라 R-CNN 구현

- 여러 데이터에 대해 R-CNN 실습 과정을 구현 및 결과 분석
- 실패한 예측 및 이상한 결과에 대한 원인 분석

실습 #2) R-CNN의 문제점을 해결하는 자신만의 아이디어 구현

- R-CNN의 문제점을 해결하는 알고리즘을 구현 하시오.
- 해결 가능한 문제점
 - ✓ 이미지 전체를 탐지
 - ✓ 객체 예측 실패
 - ✓ 하나의 객체에 여러 Bounding Box
- **아이디어 예시**
 - ✓ Selective Search 시, Bounding Box의 크기가 일정 크기 이상인 경우 해당 Bounding 박스 삭제 or 일정 크기 이하인 경우에만 Bounding Box 저장
→ 이미지 전체 탐지 방지
 - ✓ Warping 시 객체의 왜곡 발생 → Warping 된 이미지를 활용하여 CNN 학습
 - ✓ Bounding Box가 일정 비율 이상 겹치는 경우 하나만 남기고 Box 삭제

■ 속도가 느림

- R-CNN은 CNN을 실행하기 전 Selective Search 과정을 거치기 때문

■ Background error 잦음

- 배경에 객체가 있다고 판단하는 오류
- R-CNN은 Selective Search 알고리즘을 통해 제안된 영역만을 사용하기 때문에 배경을 객체로 탐지하는 오류 발생

■ Crop시 이미지 왜곡 발생

- 이미지를 crop하는 과정에서 형태가 일그러져서 분류성능 저하



해당 문제들을 해결하기 위해 YOLO 탄생

1. YOLO는 **한번의 Neural Network 실행** 만으로 해결됨. (1-stage Detector)
2. YOLO는 이미지 전체를 이용하여 예측하기 때문에 **Background error가 작음.**
3. YOLO는 이미지 전체를 이용하여 예측하기 때문에 **이미지가 왜곡되지 않음.**